

Modularity Makes Simulated Dexterous Hands Learn Faster, Not Better

Anonymous authors (double-anonymous review)

Abstract—Modular motion models such as MotionBricks build motion from reusable parts, which makes them attractive for dexterous hands, whose tasks are long and whose data are scarce. Modularity promises a hand that learns faster and reaches further. Unfortunately, prior work on synergies, latent action spaces and skill chaining changes several things at once, so the share of any gain that is due to modularity is unknown. In contrast, we isolate each of MotionBricks’ two layers on simulated hands and change one thing at a time, with the main comparisons registered before they ran. The key observation is that a flat policy that knows its stage represents any solution a modular policy represents: modularity changes how a solution is found, not which solutions exist. At the motion level, a keyframe interface with a learned in-between learner far faster than a continuous latent with the same decision rate (0.59 against 0.22 success on held-out skill sequences at 1.5 M frames), and the two are level by 5 M frames (0.49 against 0.44). At the skill level, separately trained skills do no better than the best flat policy trained on the same 25 human demonstrations (45 against 61 of 400 test episodes); the gain comes from selecting the best policy for each stage (97 against 61), which works on flat policies. Modularity therefore buys sample efficiency, not a higher ceiling. Four conditions state when it would pay.

Index Terms: Dexterous Manipulation, Reinforcement Learning, Learning from Demonstration, Multifingered Hands, Deep Learning Methods.

I. INTRODUCTION

Dexterous manipulation tasks are long and their demonstrations are scarce, so a learner that reuses parts is attractive. MotionBricks [1] generates character motion from reusable parts at two layers: poses joined by a learned in-between learner, and smart primitives that act. Modularity of this kind promises two things. A learner that works in modules should learn faster, because each module is simpler than the whole task, and it should reach further, because modules recombine into behaviour never seen whole.

Whether modularity keeps these promises in control is unclear. Synergy and latent action spaces report faster learning, but rarely a higher final success [2]–[6]. Skill chaining works on dexterous hands, but with engineered hand-offs and extra machinery [7], [8]. Each of these changes the prior, the decision rate, the data or the training procedure together, so the share of the gain that is due to modularity is unknown.

The key observation of this paper is that, in reinforcement learning, a flat policy that knows which stage it is in, given enough capacity, represents any solution that a modular policy represents. Modularity therefore cannot change which solutions exist; it changes only how they are found. This predicts that modularity speeds up learning without raising its ceiling, and that any skill-level gain comes from the search rather than from the modules themselves. We test both predictions on MotionBricks’ two layers in simulation, changing one thing at a time.

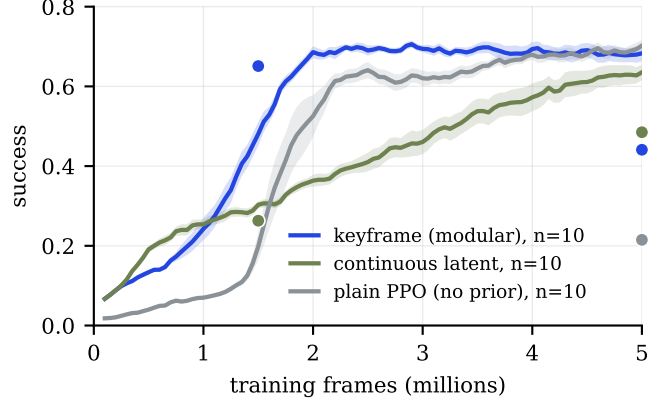


Fig. 1. Faster, not better. The modular keyframe interface (blue) reaches its final success on the training sequences by 2 M frames, while plain PPO (grey) and the continuous latent (green) take longer; by 5 M frames all three are at the same level. Lines: training success, mean and standard error over 10 RL seeds. Dots: success on held-out sequences for the same seeds at 1.5 M and 5 M frames (Section IV reports all 20 seeds at 1.5 M).

At the motion level, a keyframe interface in MotionBricks’ sense learns held-out skill sequences almost three times as well as a continuous latent that decides at the same rate, and the latent catches up later (Fig. 1). At the skill level, skills trained separately do no better than a *flat* policy, one policy for the whole task; the gain comes from selecting the best policy for each stage (Fig. 3).

Our contributions are:

- 1) **Sample efficiency, not a higher ceiling.** With the decision rate held fixed, the keyframe interface learns far faster than a continuous latent (pre-registered) and than plain PPO (post hoc), and all three end at the same level on the training sequences (Section IV).
- 2) **Selection, not separate training.** A design that separates the two at the skill level locates the gain in selecting each stage’s policy on its own, which works best on flat policies (Section V).
- 3) **Four conditions** under which modularity would pay, which account for both results and for where MotionBricks succeeds (Section VI).

II. RELATED WORK

Priors as action spaces. A learned motion prior turns a high-dimensional hand into a smaller search space. LAMP [9] uses a continuous latent prior as the action space for residual RL, and DexSynRefine [10] grounds motion primitives with residual RL. Synergy, latent and dynamical-system action spaces mostly speed up early learning [2]–[6], yet PPO on raw actions ends as high or higher once it converges [6]; reported gains in final success come where

the joint-space baseline has not converged [11], [12]. Learned skill priors [13] and action chunking [14] shorten the decision horizon in the same way as a keyframe interface. What this line of work leaves open is how much of the speed-up is the modular structure and how much is the decision rate; the comparison here holds the decision rate fixed.

Modular motion and skill chaining. Modularity appears as authored primitives over a latent backbone [1], switching dynamical systems [15], [16] and mode-adaptive gating [17]. Skill chaining trains each skill to start where the previous one ends [18], and terminal-state regularisation helps the next skill take over [8]; on dexterous hands, chaining succeeds when the hand-offs are engineered [7]. These methods add machinery at the hand-offs, so they do not show whether the gain comes from training modules separately. Analyses of hierarchical RL trace most of its benefit to exploration [19] and its efficiency to sub-problems that repeat [20]; both point to the search rather than to what a hierarchy represents.

Policy selection. Imitation-learned policies vary widely across seeds and checkpoints, and the choice among them matters as much as how they are trained [21], [22]. That work selects whole policies. Selecting per stage, the option that modular deployment opens, is not studied there.

III. HAND-MOTION PRIORS

All priors are trained on OakInk [23] hand motion, represented as 27 anatomical degrees of freedom (four per finger, five for the thumb, six for the wrist) projected from MANO [24]. The *continuous latent* is a variational autoencoder ($\beta = 1$ with free bits [25]) that encodes a 32-frame window into a 12-D latent per frame, with GRU encoder and decoder (2.28 M parameters) and a bounded output, so joint limits hold by construction. The *keyframe in-betweener*, the MotionBricks-style prior, is a GRU trained on the same motion to fill the frames between two target poses. The *clip bank* is a modular bank of $K = 16$ motion primitives mixed softly with a style code.

IV. MOTION LEVEL: FASTER, NOT BETTER

Interfaces. Each prior becomes the action space of PPO [26], and each prior interface takes one decision every 16 frames. (a) Clip bank: a soft mixture of primitives and a style. (b) Continuous latent: the 12-D latent, held for 16 frames. (c) Keyframe, in MotionBricks’ sense [1]: a target pose, reached by the learned in-betweener. (d) The same target pose reached by straight-line interpolation, with no learned prior. Each adds a per-frame residual of at most 3° , capped at $\pm 15^\circ$ in total. (e) Plain PPO sets joint targets every frame, with no prior.

Task. On the Shadow Hand E3M5 [27] in MuJoCo [28], a 2.5 cm, 0.20 kg cube rests on the palm under a lateral tilt of 1.5 g. An episode is a sequence of skills of forty frames each: hold under an x tilt, hold under a y tilt, release. Policies train on the four two-skill sequences and are scored on the two three-skill sequences they never see, over 100 episodes with the deterministic policy.

Training. PPO runs 16 parallel environments with rollouts of 256 steps, batches of 1024, learning rate 3×10^{-4} , entropy

TABLE I
EACH PRIOR AS A POLICY’S ACTION SPACE: SUCCESS ON SKILL SEQUENCES NEVER SEEN IN TRAINING. THE KEYFRAME INTERFACE LEADS AT 1.5 M FRAMES; BY 5 M THE CONTINUOUS LATENT HAS CAUGHT UP. SEEDS: 20 AT 1.5 M, 10 FOR THE STRAIGHT-LINE AND BANK ROWS AND AT 5 M.

interface	1.5 M frames	5 M frames
plain PPO (no prior)	–	0.22
continuous latent	0.22	0.49
keyframe, in-betweener	0.59	0.44
keyframe, straight-line	0.00	–
$K = 16$ clip bank	0.08	–

coefficient 0.01, 10 epochs per update, $\gamma = 0.99$, $\lambda = 0.95$ and clip 0.2. Actor and critic have two hidden layers of 256 units; exploration is state-dependent, resampled every 8 steps, from an initial log standard deviation of -1 . The reward is one unit for every frame the cube is held during a hold, and a graded per-frame reward during release. Every interface uses the same settings.

Faster. At 1.5 M frames the keyframe interface succeeds on 0.59 of held-out sequences against the latent’s 0.22 (20 seeds, t -test $p < 0.001$, 17 of 20 seeds, sign test $p = 0.003$, as declared), and the lead holds on longer unseen sequences of four and five skills (0.51 and 0.48 against 0.22 and 0.15). Fig. 1 shows the same lead on the training sequences: the keyframe interface reaches its plateau of 0.69 by 2 M frames, plain PPO comes within 0.05 of it at 2.5 M, and the latent is still below it at 5 M (0.64). Over the first 2.5 M frames its mean training success is 0.40, against 0.25 for plain PPO and 0.27 for the latent (10 seeds each, Mann–Whitney $p = 10^{-4}$ for both, post hoc). Because the latent also takes one decision every 16 frames, the difference is not the decision rate. The same target poses reached by straight lines never learn to release (0 of 10 seeds), so on this task the learned in-betweener, not the keyframes alone, carries the gain. Fig. 2 shows one held-out episode.

Not better. The lead closes (Fig. 1, Table I). At 5 M frames the latent succeeds on 0.49 of held-out sequences against the keyframe interface’s 0.44 ($p = 0.58$, 10 seeds). The keyframe interface itself falls from 0.65 at 1.5 M to 0.44 on the same seeds: with longer training both learned priors settle on holding perfectly (0.96 to 1.00 per hold on the training sequences) and releasing less than half the time. The keyframe interface still beats plain PPO on held-out sequences (0.44 against 0.22, 9 of 10 seeds, as declared). On three other objects (a heavier cube, a sphere and a cylinder) the lead is smaller and not significant (pooled test, $p = 0.067$). The clip bank is the weakest learned prior (0.08).

V. SKILL LEVEL: SELECTION, NOT SEPARATE TRAINING

Task. MotionBricks’ other layer composes smart primitives, units that act rather than poses. The test uses Adroit’s relocate [29], extended into a sparse multi-stage task: grasp the ball from the table, then carry it to a random target in the air, once or twice. The data are the 25 human demonstrations

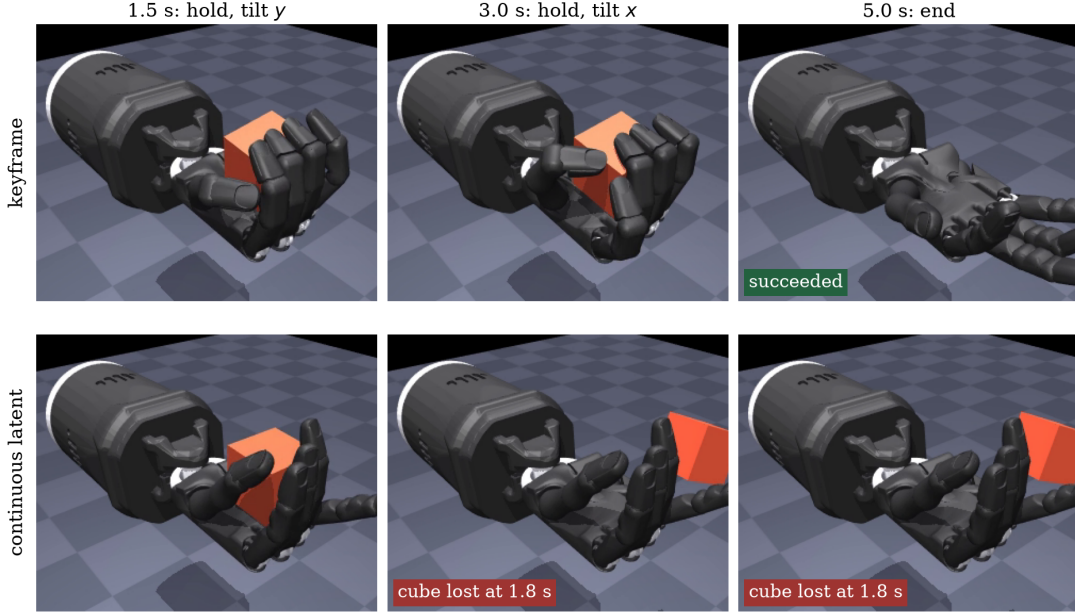


Fig. 2. The keyframe interface holds the cube through an unseen sequence; the continuous latent drops it. One held-out sequence at 1.5M frames (hold under a y tilt, hold under an x tilt, release); top, the keyframe interface; bottom, the continuous latent, which loses the cube during the first hold. RL seed 13, whose success is closest to the 20-seed means; the first evaluation episode (of 100) in which the keyframe interface succeeds and the latent fails.

of D4RL [30], each containing one grasp and one carry, and every policy is behaviour-cloned from them; PPO from scratch never completes a carry (5M steps, two seeds). A modular policy trains one skill per stage and chains them with a fixed sequencer; a flat policy is one policy for the whole task, with the stage index in its input. A stage counts as done only after its goal holds for 25 consecutive steps; with a one-step criterion, carry policies learn to fling the ball through the target, and the next skill cannot recover it [8].

Design. The design separates training from selection, on fresh seeds and 400 test episodes never used for any choice, with 20 trainings per side. Variant A is the best of 20 flat policies, used for every stage. Variant B takes the same 20 flat policies and uses the best grasper for the grasp and the best carrier for every carry. Variant C trains 10 grasp skills (on the grasp segments of the demonstrations) and 10 carry skills (on the carry segments) and chains the best of each. A against B changes only the selection; B against C changes only the training. Every choice uses 200 validation episodes per stage. Behaviour cloning regresses the policy’s mean action onto the demonstrated actions (mean squared error, Adam, learning rate 10^{-3} , batch 256, 20 epochs), with the same network as above and observation normalisation fitted to the demonstrations. An earlier registered comparison with five seeds per variant is at the floor at its declared length of four stages ($p = 0.45$); at two stages the modular variant wins (38 against 8 of 200, $p < 10^{-5}$), which we traced after the fact to per-stage selection. This design tests that reading.

Selection wins. Per-stage selection beats the best flat policy: 97 against 61 of 400 episodes at two stages ($p = 0.002$) and 47 against 12 at three ($p < 10^{-5}$). It wins through

TABLE II
TRAINING OR SELECTION: 400 FRESH TEST EPISODES, PRE-REGISTERED. PER-STAGE SELECTION (B) COMPLETES THE MOST EPISODES, THROUGH THE CARRY. PER-STAGE RATES ARE FOR THE TWO-STAGE TASK (CARRY | GRASP: CARRIES COMPLETED AFTER A COMPLETED GRASP); THE LAST TWO COLUMNS COUNT EPISODES THAT COMPLETED EVERY STAGE. FISHER’S EXACT TEST AGAINST VARIANT A.

variant	grasp	carry grasp	2 stages	3 stages
A: best flat policy	0.77	0.20	61	12
B: best flat per stage	0.72	0.34	97 ($p = 0.002$)	47 ($p < 10^{-5}$)
C: separate skills	0.57	0.20	45 ($p = 0.12$)	6 ($p = 0.23$)

the carry: B’s carrier completes 0.34 of carries after a grasp, against A’s 0.20.

Separate training does not. The registered prediction, that separately trained skills beat the best flat policy, fails (45 against 61, $p = 0.12$), and separate training loses to per-stage selection (45 against 97, $p < 10^{-5}$). C loses at the grasp (0.57 against 0.77). Without selection, the declared comparison over seeds finds separately trained carry skills worse than the carry stage of flat policies (0.14 against 0.27 on validation, $p < 10^{-5}$) and grasp skills not distinguishable ($p = 0.19$); on the test episodes both are lower (grasp 0.14 against 0.23, carry 0.17 against 0.26, post hoc).

Not a matter of pool size. B chooses from 20 policies per stage and C from 10, and each variant is one selected combination. A post hoc follow-up, with its analysis written down before it ran, scores every selection rule by the product of its per-stage test rates. Restricted to each of the 184756 possible pools of 10 flat policies, per-stage selection still

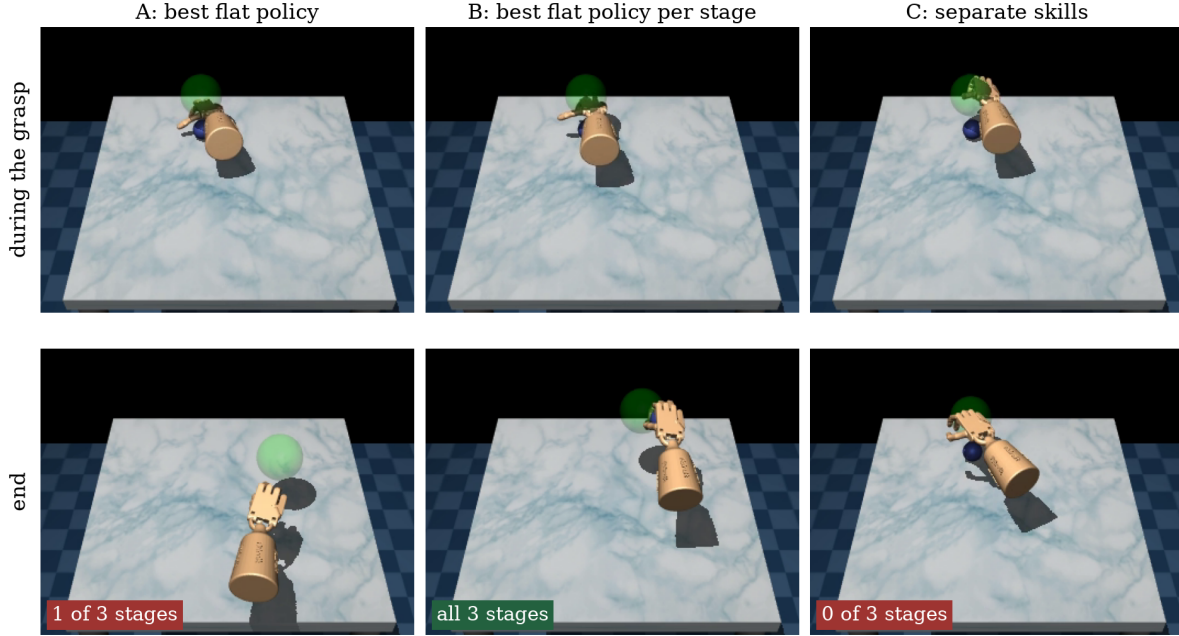


Fig. 3. Selecting per stage completes the task where the other variants stop. Same test start for the three variants (grasp, carry, carry again); top, during the grasp; bottom, at the end. B, the best flat policy for each stage, completes all three stages; A grasps the ball but never delivers it, and C never completes the grasp. One test episode, the first in which B completes every stage; B does so in 47 of 400 test episodes (Table II).

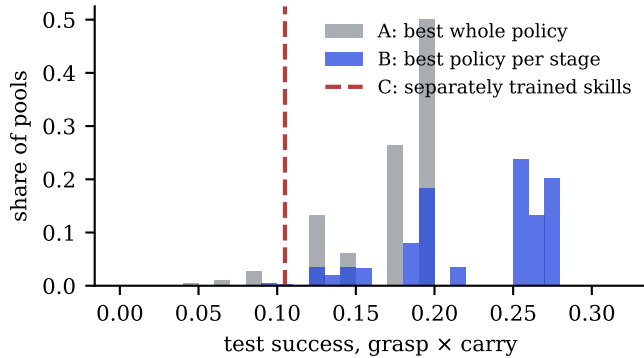


Fig. 4. Selection, not pool size. With pools matched at 10 policies per stage, per-stage selection (B, blue) lies above separate training (C, dashed line) in 99% of pools and mostly above whole-policy selection (A, grey). Each bar counts pools of 10 flat policies (184 756 in all) by the product of per-stage test rates (grasp $\times$ carry), with every choice made on validation.

beats separate training in 99% of pools (mean 0.23 against 0.10). Over 10 000 redrawn pools of 10 per side, it beats separate training in 96% of draws and whole-policy selection in 82% (Fig. 4).

VI. DISCUSSION: WHEN DOES MODULARITY PAY?

Both results follow from the key observation. A flat policy that knows its stage represents the modular solution, so modularity cannot raise the ceiling; it changes the search. At the motion level the change is faster exploration; at the skill level it is the option to select each stage’s policy on its own. In RL, modular versus flat is therefore a comparison of training and deployment procedures, not of architectures,

TABLE III

THE FOUR CONDITIONS AGAINST THE TWO LAYERS TESTED. EACH LAYER FAILS AT LEAST ONE; NEITHER MEETS (IV).

condition	motion level	skill level
(i) solution inside	partly: the grip needs 83°, OakInk and TACO [31] reach 73–76°; the residual covers the gap	yes: skills learned on the task
(ii) clean hand-offs	yes: poses only	only when enforced: a one-step stage lets carry fling the ball
(iii) something to amortise	partly: faster exploration, one task	yes: PPO from scratch never carries
(iv) flat cannot match	no: caught up at 5 M	no: flat policies selected per stage do best

consistent with the finding that most of the benefit of hierarchy is better exploration [19].

A chain of modules succeeds with probability $\prod_i P(\text{module } i \mid \text{end states of module } i-1)$, so every benefit of modularity is paid for at the hand-offs. Four conditions, formulated during this study and offered as hypotheses, state when modularity pays: (i) the solution lies inside what the modules express; (ii) hand-offs are clean; (iii) there is something to amortise, such as a horizon a flat learner cannot cross or combinations it has not seen; and (iv) a flat learner given the same data does not get the same benefit. Table III maps the results onto them; each layer tested fails at least one.

The motion-level result matches what synergy and latent action spaces report, faster early learning with an equal

or lower final success [2]–[6]; higher final success appears where the joint-space baseline has not converged [11], [12]. Skill chaining on dexterous hands succeeds when the hand-offs are engineered [7], which is condition (ii).

At the skill level, what survives is that each stage is validated and selected on its own. With N trained policies, per-stage selection picks the best of each stage, where whole-policy selection picks only the best product. If a carrier’s success does not depend on which grasper precedes it, the product of the per-stage best is never below the best whole policy. This benefit belongs to modular *deployment*, not modular training: it is largest with flat policies. We have not found it framed as a benefit of modularity before; work on policy selection picks whole policies [21], [22], and work on skill chaining credits the gain to training [7], [8].

MotionBricks meets all four conditions in animation: its goal is human-like motion and its data are human motion, it is kinematic so any pose starts the next brick, one library serves many applications, and it has 10^5 clips. The dexterous layers tested here each fail at least one.

VII. LIMITATIONS AND FUTURE WORK

Everything here is in simulation, with one task per layer, and the early lead is significant on the cube only. Three extensions would test how far the findings generalise. First, more tasks, hands and objects, and real hardware. Second, stronger modular baselines: learned skill priors, action chunking and hand-off methods such as T-STAR, which the comparison here does not include; the skill level uses behaviour cloning from 25 demonstrations and a fixed sequencer. Third, a positive test of the four conditions, which are hypotheses formulated during this study: a task that meets all four should show a modular gain in the ceiling, not only in speed.

VIII. CONCLUSION

Modularity makes these simulated dexterous hands learn faster, not better. Because a flat policy that knows its stage represents any modular solution, modularity changes the search and not the ceiling. A MotionBricks-style keyframe interface buys sample efficiency that a continuous latent later matches, and separately trained skills do no better than one flat policy. Where modularity helps at the skill level, it is by selecting the best policy for each stage, which works on flat policies. Before building a modular policy, check the four conditions, starting with the hand-offs.

ACKNOWLEDGMENT

Generative AI (Claude, Anthropic) was used under the authors’ direction to draft and revise the text, write the analysis scripts, and launch the experiment runs. The authors designed the study and take full responsibility for the content. Every number in the paper is computed by scripts from the run artifacts; the code, the artifacts and the pre-registration log will be released on acceptance.

REFERENCES

- [1] T. Wang et al., “MotionBricks: Scalable Real-Time Motions with Modular Latent Generative Model and Smart Primitives,” ACM TOG 45(4), SIGGRAPH 2026, doi:10.1145/3811334, arXiv:2604.24833.
- [2] C. Berg, V. Caggiano, V. Kumar, “SAR: Generalization of Physiological Agility and Dexterity via Synergistic Action Representation,” RSS 2023, arXiv:2307.03716.
- [3] V. Caggiano, S. Dasari, V. Kumar, “MyoDex: A Generalizable Prior for Dexterous Manipulation,” ICML 2023, arXiv:2309.03130.
- [4] E. Y. Puang, F. Ceola, G. Pasquale, L. Natale, “PCHands: PCA-based Hand Pose Synergy Representation on Manipulators with N-DoF,” IEEE-RAS Humanoids 2025, arXiv:2508.07945.
- [5] H. Yuan, B. Zhou, Y. Fu, Z. Lu, “Cross-Embodiment Dexterous Grasping with Reinforcement Learning,” arXiv:2410.02479, 2024.
- [6] S. Bahl, M. Mukadam, A. Gupta, D. Pathak, “Neural Dynamic Policies for End-to-End Sensorimotor Learning,” NeurIPS 2020, arXiv:2012.02788.
- [7] Y. Chen, C. Wang, L. Fei-Fei, C. K. Liu, “Sequential Dexterity: Chaining Dexterous Policies for Long-Horizon Manipulation,” CoRL 2023, arXiv:2309.00987.
- [8] Y. Lee, J. J. Lim, A. Anandkumar, Y. Zhu, “Adversarial Skill Chaining for Long-Horizon Robot Manipulation via Terminal State Regularization,” CoRL 2021, arXiv:2111.07999.
- [9] X. Yang et al., “LAMP: Latent Motion Prior-Guided Real-World Learning for Dexterous Hand Manipulation,” arXiv:2607.06323, 2026.
- [10] H. Lee, H. Jung, S.-H. Heo, S. Yang, “DexSynRefine: Synthesizing and Refining Human-Object Interaction Motion for Physically Feasible Dexterous Robot Actions,” arXiv:2605.05925, 2026.
- [11] P. Rivera, E. Valarezo Añazco, T.-S. Kim, “Object Manipulation with an Anthropomorphic Robotic Hand via Deep Reinforcement Learning with a Synergy Space of Natural Hand Poses,” Sensors 21(16):5301, 2021, doi:10.3390/s21165301.
- [12] Z. Luo, J. Cao, S. Christen, A. Winkler, K. Kitani, W. Xu, “Omnigrasp: Grasping Diverse Objects with Simulated Humanoids,” arXiv:2407.11385, 2024.
- [13] K. Pertsch, Y. Lee, J. J. Lim, “Accelerating Reinforcement Learning with Learned Skill Priors,” CoRL 2020, arXiv:2010.11944.
- [14] T. Z. Zhao, V. Kumar, S. Levine, C. Finn, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” RSS 2023, arXiv:2304.13705.
- [15] S. Linderman, M. Johnson, A. Miller, R. Adams, D. Blei, L. Paninski, “Bayesian Learning and Inference in Recurrent Switching Linear Dynamical Systems,” AISTATS 2017, PMLR 54:914–922.
- [16] Z. Dong, B. A. Seybold, K. Murphy, H. H. Bui, “Collapsed Amortized Variational Inference for Switching Nonlinear Dynamical Systems,” ICML 2020, PMLR 119:2638–2647.
- [17] H. Zhang, S. Starke, T. Komura, J. Saito, “Mode-Adaptive Neural Networks for Quadruped Motion Control,” ACM TOG 37(4), Article 145, SIGGRAPH 2018.
- [18] G. Konidaris, A. Barto, “Skill Discovery in Continuous Reinforcement Learning Domains using Skill Chaining,” NIPS 22, pp. 1015–1023, 2009.
- [19] O. Nachum, H. Tang, X. Lu, S. Gu, H. Lee, S. Levine, “Why Does Hierarchy (Sometimes) Work So Well in Reinforcement Learning?,” NeurIPS 2019 Deep RL Workshop, arXiv:1909.10618.
- [20] Z. Wen, D. Precup, M. Ibrahimi, A. Barreto, B. Van Roy, S. Singh, “On Efficiency in Hierarchical Reinforcement Learning,” NeurIPS 2020.
- [21] A. Mandlkar et al., “What Matters in Learning from Offline Human Demonstrations for Robot Manipulation,” CoRL 2021, arXiv:2108.03298.
- [22] V. Kurenkov, S. Kolesnikov, “Showing Your Offline Reinforcement Learning Work: Online Evaluation Budget Matters,” ICML 2022, arXiv:2110.04156.
- [23] L. Yang et al., “OakInk: A Large-scale Knowledge Repository for Understanding Hand-Object Interaction,” CVPR 2022, arXiv:2203.15709.
- [24] J. Romero, D. Tzionas, M. J. Black, “Embodied Hands: Modeling and Capturing Hands and Bodies Together,” ACM TOG 36(6), Article 245, SIGGRAPH Asia 2017.
- [25] I. Higgins et al., “beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework,” ICLR 2017.
- [26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.

- [27] K. Zakka, Y. Tassa, and MuJoCo Menagerie Contributors, “MuJoCo Menagerie: A Collection of High-Quality Simulation Models for MuJoCo,” 2022. https://github.com/google-deepmind/mujoco_menagerie
- [28] E. Todorov, T. Erez, Y. Tassa, “MuJoCo: A Physics Engine for Model-Based Control,” IROS 2012, pp. 5026–5033.
- [29] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, S. Levine, “Learning Complex Dexterous Manipulation with Deep Reinforcement Learning and Demonstrations,” RSS 2018, arXiv:1709.10087. Environments as packaged in Gymnasium-Robotics 1.4.2.
- [30] J. Fu, A. Kumar, O. Nachum, G. Tucker, S. Levine, “D4RL: Datasets for Deep Data-Driven Reinforcement Learning,” arXiv:2004.07219, 2020.
- [31] Y. Liu, H. Yang, X. Si, L. Liu, Z. Li, Y. Zhang, Y. Liu, L. Yi, “TACO: Benchmarking Generalizable Bimanual Tool-Action-Object Understanding,” CVPR 2024, pp. 21740–21751, arXiv:2401.08399.